



FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

INFORME BASE DE DATOS

*Ecoflor: Base de datos segura, automatizada e
inteligente para la gestión de datos*

Autores:

Chávez Díaz Jhajaira Roxana (100% participación)
Lezcano Moreno Mariana Jasmin (100% participación)
Portal Carrión Mariela (100% participación)
Sandoval Villanueva Betsy Yomar (100% participación)
Yopla Huamán Ana Elizabeth (100% participación)

Curso:

Base de Datos Avanzada y Big Data-

Docente:

Marlon Joer Linarez Vigo

Cajamarca - Perú

2026-2

Cooperativa de Rosas Ecoflor

Repositorio: https://github.com/jasminlezcana07-commits/Cooperativa_Rosas.git

1.1 Relevamiento, supuestos y articulación curricular

Fecha de levantamiento: 29-08-2026.

Fuente: Datos recolectados por medio de una entrevista directa con los socios Francisco Cueva y Zoila Huamán y dataset creados con Claude, autorizado por ellos, ya que no llevan registro digital (ni Excel).

Contexto: Cooperativa de 2 socios en el caserío de Llagamarca-Baños del Inca-Cajamarca, 2 viveros cercanos entre sí, 4 trabajadores que rotan entre ambos viveros, aproximadamente cosecha 3 veces por semana (entre 20-120 paquetes por día, con 24 unidades cada paquete), 4 colores de rosa(Rojo, Blanco, Amarillo, Rosado), 3 rangos de precio por tamaño de tallo($\leq 50\text{cm}$, 60cm, 70-90 cm). Venden a 3 clientes mayoristas en Chiclayo(Son simulados), coordinados mediante WhatsApp.

Tabla 1: *Esquema de datos fuente*

Archivo	Campos	Volumen
cosechas_raw.csv	id_cosecha, fecha, vivero, color, tamaño_cm, cantidad_paquetes, trabajador	246 filas
ventas_raw.csv	id_venta, fecha, cliente, color, tamaño_cm, cantidad_paquetes, precio_unitario_paquete	30 filas

Tabla 2: *Problemas de calidad identificados*

Problema	Registros afectados	Ejemplo
Duplicados exactos	6	Fila de cosecha repetida
Nulos en cantidad_paquetes	17	Campo vacío
Colores mal escritos (12 variantes de 4 colores)	~15% de filas	"ROJO", "Rojoo", "Blanc0", "amarillo "

Precios inconsistentes con el tallo	3 de 30 ventas	Tallo 50cm cobrado a S/25 en vez de S/20
-------------------------------------	----------------	--

Supuestos declarados

1. Simulación por cosecha dentro del rango de 20-120 paquetes por día, se simula dentro del rango que dieron los socios. Esto Permite dimensionar las pruebas y el procesamiento de datos.
2. Cada paquete contiene un único color y tamaño de tallo sin ningún tipo de excepciones. Esto simplifica el modelado sin afectar su validez.
3. Se utilizan nombres inventados (Mayorista Chiclayo Norte, Distribuidora Flor del Valle y Comercial Rosas SAC) para proteger los datos de la cooperativa. Los socios autorizan el uso de IA para generar datos adicionales, evitando exponer información.
4. Los pedidos se reciben en formato libre, lo que nos permite hacer uso de NoSQL para gestionar los datos de forma libre.

Requerimientos funcionales

Tabla 3: *Requisitos funcionales y criterios de éxito verificable*

RF	Descripción	Criterio de éxito
RF1	Registrar cosecha sin cantidades negativas/nulas.	<i>sp_RegistrarCosecha</i> rechaza y loguea intentos inválidos
RF2	Calcular precio automáticamente por tallo.	<i>fn_PrecioPorTamano</i> acierta en los 3 rangos (≥ 10 pruebas)
RF3	Trazabilidad de quién registró cada operación.	<i>LogAuditoria</i> tiene 1 registro por INSERT en Cosechas/Ventas
RF4	Consultar venta por cliente/color en < 3 pasos.	El dashboard responde con drill-down

Articulación curricular

Este proyecto hace uso de saberes previos de Base de Datos, como SQL, procedimientos almacenados, normalización e integridad referencial. Del mismo modo, prepara al equipo para Ciencia de Datos, mediante el modelado dimensional y los procesos ETL, y para Ingeniería de Software, mediante el diseño de una arquitectura híbrida y orientada a datos.

1.2 Automatización mediante objetos de base de datos

La automatización se implementó en SQL Server mediante tablas, una función escalar, procedimientos almacenados y triggers, con el propósito de centralizar las reglas de negocio, controlar la integridad de los datos y mantener la trazabilidad de las operaciones.

Esquema de base de datos

El modelo contempla las tablas *Viveros*, *Trabajadores*, *Colores*, *Clientes*, *Paquetes*, *Cosechas*, *Ventas* y *LogAuditoria*. La tabla *Paquetes* establece los rangos de longitud de tallo y sus precios oficiales: 50 cm = S/ 20, 60 cm = S/ 23 y 70–90 cm = S/ 26. El script completo se encuentra en el repositorio, en 01_automatizacion/01_esquema_procedimientos_triggers.sql.

Función escalar

La función *fn_PrecioPorTamano* permite obtener automáticamente el precio correspondiente a una determinada longitud de tallo, consultando el catálogo de paquetes. De esta manera se mantiene centralizada la regla de negocio.

```
sql

CREATE FUNCTION fn_PrecioPorTamano (@tamano_cm INT)

RETURNS DECIMAL(6,2)

AS BEGIN

    RETURN (SELECT precio_unitario FROM Paquetes WHERE

tamano_cm = @tamano_cm);

END;
```

Procedimiento 1 - sp_RegistrarCosecha

Valida que cantidad_paquetes no sea nula ni negativa, valida que el color y el tallo existan en el catálogo, y usa BEGIN TRY/CATCH + BEGIN TRANSACTION + SAVE TRANSACTION + ROLLBACK en caso de error, registrando el error en LogAuditoria.

Procedimiento 2 - sp_RegistrarVenta

Antes de insertar, calcula el stock disponible sumando Cosechas de los últimos 7 días para ese color/tallo, y rechaza la venta si supera ese stock (control de excepción con THROW).

Se implementaron dos triggers sobre las tablas transaccionales del sistema, ambos definidos como AFTER INSERT para no interferir con la validación previa realizada por los procedimientos almacenados.

Trigger 1 - trg_AuditoriaCosecha: Se ejecuta sobre la tabla Cosechas y registra automáticamente en LogAuditoria quién realizó la cosecha, qué cantidad se registró y en qué momento.

Trigger 2 - trg_ValidarPrecioPorTallo: Se ejecuta sobre la tabla Ventas y compara el precio registrado en cada venta con el precio oficial correspondiente al rango de tallo vendido. Si ambos valores no coinciden, el trigger genera una alerta en LogAuditoria

Pruebas ejecutadas

Las pruebas contemplan operaciones válidas y casos de error, como el registro de una cosecha con cantidad negativa y una venta que supera el stock disponible. La consulta de LogAuditoria permite verificar los resultados y comprobar que las operaciones y excepciones sean registradas correctamente.

```
sql
```

```
EXEC sp_RegistrarCosecha '2026-01-05', 1, 'Rojo', 60,  
15, 1;
```

```
EXEC sp_RegistrarVenta '2026-01-06', 1, 'Rojo', 60,  
10;
```

```
EXEC sp_RegistrarCosecha '2026-01-05', 1, 'Rojo', 60,
-5, 1; -- error controlado

EXEC sp_RegistrarVenta '2026-01-06', 1, 'Rojo', 60,
999; -- excede stock

SELECT * FROM LogAuditoria ORDER BY id_log DESC;
```

Figura 1

Registro de auditoría de las operaciones realizadas en la base de datos

	id_log	tabla_afectada	operacion	usuario_bd	fecha_hora	detalle
1	3	Ventas	ERROR_INSERT	MarielaMarie	2026-09-14 21:32:27.870	La venta supera el stock cosechado en los últimos ...
2	2	Cosechas	ERROR_INSERT	MarielaMarie	2026-09-14 21:32:24.867	La cantidad de paquetes no puede ser negativa o n...
3	1	Cosechas	INSERT	MarielaMarie	2026-09-14 21:32:20.937	Cosecha id=1 vivero=1 color_id=1 paquetes=15 tra...

Nota. La figura muestra los registros generados en LogAuditoria como resultado de las operaciones realizadas sobre la base de datos. Se evidencia tanto una operación ejecutada correctamente como dos intentos rechazados por una validación del sistema.

Comparación de alternativas

Para implementar las reglas de validación y trazabilidad se evaluaron dos alternativas: centralizar la lógica en la aplicación o implementarla directamente en la base de datos.

Tabla 4: *Comparación de alternativas*

Criterio	Lógica en aplicación	Lógica en BD (elegida)
Consistencia garantizada	Depende de que toda app valide	Se valida siempre, sin importar quién inserte
Trazabilidad automática	Hay que programarla manualmente	El trigger la registra automáticamente

Justificación: la cooperativa no tiene un solo sistema fijo (a veces se registra desde WhatsApp/app simple, a veces desde SSMS); poner la validación en la BD asegura que ningún dato inválido entre sin importar el origen.

1.3 Seguridad y cumplimiento normativo

Se implementó un esquema de seguridad basado en roles con privilegios diferenciados, con el objetivo de aplicar el principio de mínimo privilegio y controlar el acceso a la información. El script completo se encuentra en 02_seguridad/02_seguridad_roles_backup_indices.sql.

Los roles definidos son:

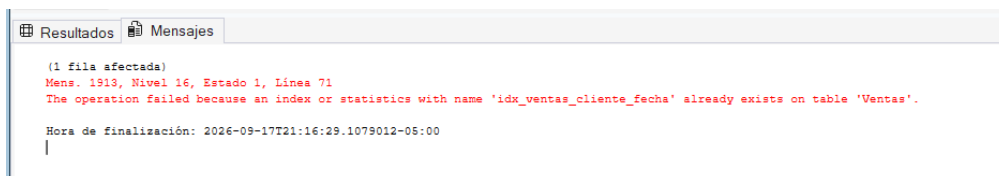
- rol_administrador (Francisco y Zoila): control total.
- rol_analista (encargado de ventas): solo ejecuta sp_RegistrarCosecha/sp_RegistrarVenta y consulta Cosechas/Ventas.
- rol_auditor (auditor externo): solo lectura de LogAuditoria, Ventas y Cosechas; DENY explícito de INSERT/UPDATE/DELETE.

Política de respaldo: Se estableció una política de copias de seguridad FULL + DIFFERENTIAL y se probó la restauración mediante RESTORE DATABASE sobre una base de datos de prueba denominada CooperativaRosas_TEST.

Análisis de rendimiento: Se evaluó una consulta de ventas por cliente y rango de fechas antes y después de implementar el índice no clúster idx_ventas_cliente_fecha, definido sobre id_cliente y fecha, con inclusión de cantidad y precio.

Figura 2

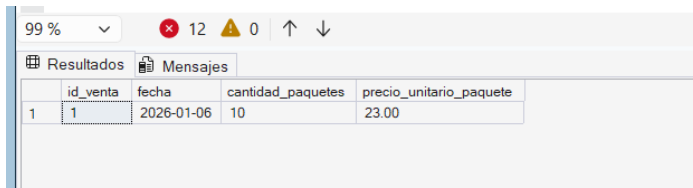
Antes del índice



Nota: Captura del plan de ejecución "antes" (Table/Index Scan) y "después"

Figura 3

Después del índice



	id_venta	fecha	cantidad_paquetes	precio_unitario_paquete
1	1	2026-01-06	10	23.00

Nota: *Captura del plan de ejecución "después" (Table/Index Scan)*

Tabla 5: Checklist normativo

Norma	Control	Estado
Ley N.º 29733, Art. 9º (Consentimiento)	Consentimiento informal de trabajadores/clientes	PENDIENTE FORMALIZAR
Ley N.º 29733, Art. 11º (Seguridad técnica)	Roles diferenciados + backup	CUMPLE
Ley N.º 29733, Art. 14º (Trazabilidad)	LogAuditoria	CUMPLE
ISO 27001, A.9 (Control de acceso)	GRANT/DENY por rol	CUMPLE
ISO 27001, A.12.3 (Copias de seguridad)	FULL + DIFFERENTIAL	CUMPLE
ISO 27001, A.12.4 (Registro de eventos)	LogAuditoria	CUMPLE

Consecuencia sobre los afectados: sin roles ni auditoría, cualquier persona con acceso podría alterar cantidades o precios sin dejar rastro, afectando el ingreso de Francisco y Zoila y la confianza de sus 3 clientes; también expondría datos de los 4 trabajadores (nombre, vivero asignado) sin control de quién los consulta.

1.4 Integración SQL–NoSQL

Colección pedidos_whatsapp (MongoDB) — CRUD documentado (script completo en repo: 03_nosql/03_mongodb_operaciones.js), con documentos de ejemplo por cada uno de los 3 clientes, incluyendo notas libres ("ramos armados", "entregar con cinta dorada") que no siempre tienen la misma forma.

Tabla 6: *Comparación SQL vs NoSQL*

Criterio	SQL Server	MongoDB
Modelo de datos	Rígido, columnas fijas (Cosechas/Ventas)	Flexible, cada pedido puede variar de campos
Escalabilidad	Vertical, suficiente para 3 clientes	Horizontal, útil si crecieran clientes
Consistencia	Fuerte (transacciones ACID)	Eventual, suficiente para notas de WhatsApp

Decisión: arquitectura híbrida. SQL Server para Cosechas/Ventas (transacciones y KPIs); MongoDB solo para el detalle variable de pedidos por WhatsApp que no requiere control transaccional estricto.

1.5 Data Warehouse y procesos ETL

Modelo dimensional (esquema estrella, Kimball). Hechos: FactVentas, FactCosechas. Dimensiones: DimTiempo, DimCliente, DimColor, DimTamano, DimVivero. DDL completo en repo: 04_datawarehouse/05_ddl_datawarehouse.sql.

ETL (Python/pandas, 04_etl_datawarehouse.py) — extrae de los CSV crudos, limpia (elimina duplicados y nulos, normaliza colores y precios), enriquece (ingreso_total) y carga las dimensiones + tablas de hechos.

Log real de ejecución

Extraídos 246 registros de cosechas y 30 de ventas (datos crudos)

Duplicados eliminados en Cosechas: 6

Filas con cantidad_paquetes nula eliminadas: 17

Colores normalizados a: Rojo, Amarillo, Blanco, Rosado

Precios inconsistentes corregidos: 2

Enriquecimiento: columna 'ingreso_total' calculada

FactVentas construida con 30 filas

FactCosechas construida con 223 filas (usa DimVivero)

246 → 223 filas de cosecha tras limpieza. FactVentas: 30 filas, ingreso total S/ 24,395 sobre 1,060 paquetes vendidos.

Kimball vs Inmon

1. **Rapidez de implementación:** la cooperativa necesita un dashboard simple y rápido (bottom-up), no un modelo corporativo completo.
2. **Orientación a preguntas de negocio concretas:** ventas por color, cliente y vivero — enfoque bottom-up frente al top-down de Inmon.

1.6 Dashboard BI, KPIs y análisis OLAP

Archivo: 05_dashboard/dashboard.Power Bi (Power Bi + Chart.js, alternativa open-source exportable a Power BI/Metabase).

Tabla 7: *KPIs con gobernanza*

KPI	Fórmula	Fuente	Responsable	Frecuencia
Ingreso total por color	SUM(ingreso_total) por color	FactVentas + DimColor	Zoila Huamán	Semanal
Paquetes vendidos por cliente	SUM(cantidad_paquetes) por cliente	FactVentas + DimCliente	Francisco Cueva	SemanalHC
Ingreso total por semana	SUM(ingreso_total) por semana	FactVentas + DimTiempo	Francisco Cueva	Semanal

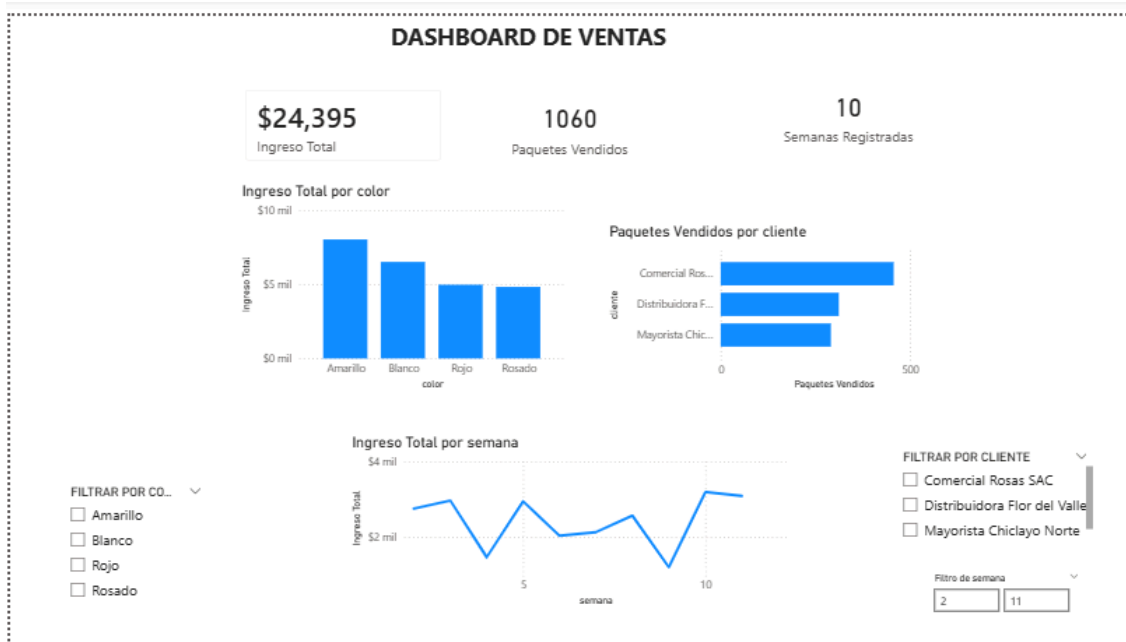
Operación OLAP (drill-down): desde "ingreso total por color" se hace clic en una barra para ver "ingreso por color y cliente" (ej. Rojo → Comercial Rosas SAC: S/1,081).

Preguntas de negocio respondidas: ¿qué color genera más ingresos? ¿cuál de los 3 clientes compra más paquetes?

Gobernanza: completitud (17 filas nulas descartadas antes del DW), consistencia (colores normalizados, 2 precios corregidos), trazabilidad (cada INSERT queda en LogAuditoria).

Figura 4

Dashboard de inteligencia de negocios y análisis de indicadores



Nota. El dashboard presenta los indicadores de ingreso total por color, paquetes vendidos por cliente e ingreso total por semana.